

WEEK 6

Week 6: Parallel Programming - Lab Guide

HPC & Cluster Computing

Objective: Write, compile, and debug multi-threaded C applications using OpenMP.

Context: We will run these simple programs on the Master Node (or a Compute Node interactively) to understand how threads utilize the cores of a single processor.

---

1. Prerequisites

- You must have basic familiarity with `nano` and the Linux shell.
- Ensure `gcc` is installed: `gcc --version`.
- Ensure the OpenMP library is present (`libgomp`, usually standard with GCC).

---

2. Exercise 1: Hello World (Fork-Join)

We will demonstrate the creation of a team of threads.

2.1 Write the Code

Create a file named `hello_omp.c`.

```c
#include <stdio.h>
#include <omp.h> // The OpenMP Header

int main() {
    printf("Master Thread: Beginning Serial Execution...\

    // The Parallel Region - Fork!
    #pragma omp parallel
    {
        // Each thread gets its own ID
        int thread_id = omp_get_thread_num();
        int total_threads = omp_get_num_threads();

        printf("Hello World from Thread %d of %d\n", thre
    }
    // Join! Back to Master Thread only.

    printf("Master Thread: Parallel Execution Finished.\r
    return 0;
}
```

2.2 Compile and Run

We must use the `-fopenmp` flag to tell GCC to respect the pragmas.

```
gcc -fopenmp hello_omp.c -o hello_omp
./hello_omp
```

Note the output provided. How many threads ran? This likely defaults to the number of Cores on your VM.

In HPC, we explicitly control the resources. Set the environment variable `OMP_NUM_THREADS` to change the team size.

```
export OMP_NUM_THREADS=4
./hello_omp
```

Observe: Run it multiple times. Do the threads print in order (0, 1, 2, 3)? *Answer*: No. The OS scheduler decides who runs when. Parallelism is non-deterministic!

---

3. Exercise 2: The Race Condition

We will break the program to understand synchronization.

3.1 The Broken Code

Create `race_demo.c`. We want to count to 1000 using multiple friends.

```
#include <stdio.h>
#include <omp.h>

int main() {
    int count = 0;

    // Use 10 threads. Each adds 100 to the count.
    // Total should be 1000.
    #pragma omp parallel for num_threads(10)
    for(int i=0; i<1000; i++) {
        count++; // CRITICAL FAILURE: This is not atomic!
```

```
    printf("Final Count: %d (Expected: 1000)\n", count);
    return 0;
}
```

3.2 Run and Fail

```
gcc -fopenmp race_demo.c -o race_demo
./race_demo
```

Run it 5 times. You will likely see values like 998, 985, 1000, 990. *Why?* Two threads read the value `500` at the same time, both add 1 locally, and both write `501`. One increment is lost.

3.3 The Fix

Modify the code to protect the shared variable.

```
    #pragma omp parallel for num_threads(10)
    for(int i=0; i<1000; i++) {
        #pragma omp atomic
        count++;
    }
```

Recompile and run. It should consistently hit 1000.

4. Exercise 3: Calculating Pi (Numerical Integration)

A classic HPC benchmark. We approximate Pi by integrating the function $f(x) = 4/(1 + x^2)$.

4.1 Write `pi_omp.c`

```c
#include <stdio.h>
#include <omp.h>

static long num_steps = 100000000; // 100 Million steps
double step;

int main() {
    int i;
    double x, pi, sum = 0.0;
    double start_time = omp_get_wtime();

    step = 1.0 / (double) num_steps;

    // PARALLEL REGION
    // 'reduction(+:sum)' tells OpenMP to give each threa
    // then add them all together at the end safely.
    #pragma omp parallel for private(x) reduction(+:sum)
    for (i = 0; i < num_steps; i++) {
        x = (i + 0.5) * step;
        sum = sum + 4.0 / (1.0 + x * x);
    }

    pi = step * sum;
    double time_taken = omp_get_wtime() - start_time;

    printf("Pi = %.15f\n"
```

```
printf("Time: %f seconds\n", time_taken);
```

4.2 Benchmark

Compile: `gcc -fopenmp pi_omp.c -o pi_omp`

1. Run with 1 Thread (Serial): `export OMP_NUM_THREADS=1; ./pi_omp`
2. Run with 4 Threads: `export OMP_NUM_THREADS=4; ./pi_omp`

Calculate your **Speedup**: $Speedup = T_{serial} / T_{parallel}$. Ideally, 4 threads should be 4x faster (linear scaling), but overhead prevents perfection.

---

5. Submission

Submit a PDF containing:

1. Screenshot of `hello_omp` running with 4 threads.
2. Screenshot of `race_demo` failing (showing a wrong count).
3. Screenshot of `pi_omp` speedup comparison (Time for 1 thread vs 4 threads).